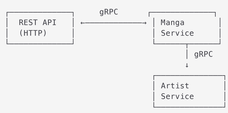

Lab: gRPC Service Implementation - Tasks

Duration: 2.5 hours | **Points:** 100

- Lab Overview
- Setup (10 minutes)
  - Install Protocol Buffers Compiler
  - Install Go Plugins
  - Create New Project
  - 📚 Study Materials
- ✍️ Task 1: Protocol Buffers Basics (15 points)
  - 🎯 Real-World Application
- ✍️ Task 2: Simple gRPC Service (20 points)
  - 🎯 Real-World Application
- ✍️ Task 3: Book Catalog Service with Database (20 points)
  - 🎯 Real-World Application
- ✍️ Task 4: Advanced Search and Error Handling (20 points)
  - 🎯 Real-World Application
- ✍️ Task 5: Complete Microservice with Multiple Services (25 points) - REQUIRED
  - 🎯 Real-World Application
- Submission Requirements (100 points)
  - Tasks (85 points)
  - Code Quality (15 points)
  - Bonus (+15 points)
- Testing Checklist
- Connection to Final Project (MangaHub)
  - What You Practiced (Books):
  - Direct Skill Transfer:
  - Code Patterns You Can Reuse:
  - What Changes for MangaHub:
  - MangaHub Architecture:
  - Next Steps for Your Project:
- Getting Help
- Common Protobuf Commands Reference

Lab Overview

Objective: Learn to build high-performance gRPC services using Protocol Buffers, implementing microservice communication patterns with type safety and automatic code generation.

What You'll Build: A complete Book Catalog microservice using gRPC that can communicate with other services - working with books to teach patterns you'll apply to manga in your project.

Why This Matters for Your Project:
The skills you learn here apply directly to your MangaHub project:

- gRPC for internal service communication (books → manga services)
- Protocol Buffers for efficient data serialization
- Unary RPC calls (single request/response)
- Error handling with status codes
- Service-to-service communication patterns
- Type-safe API contracts

Key Transferable Skills:

- ✅ Protocol Buffers message definition
- ✅ gRPC service implementation
- ✅ Unary RPC patterns
- ✅ Context and timeout management
- ✅ gRPC error handling
- ✅ Microservice communication

Setup (10 minutes)

Install Protocol Buffers Compiler

macOS:

```
brew install protobuf
```

Ubuntu/Debian:

```
sudo apt update
sudo apt install -y protobuf-compiler
```

Windows:
Download from: https://github.com/protocolbuffers/protobuf/releases

Verify installation:

```
protoc --version
# Should show: libprotoc 3.x.x or higher
```

Install Go Plugins

```
# Install protoc plugins for Go
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

# Add to PATH (add to ~/.bashrc or ~/.zshrc)
export PATH="$PATH:$(go env GOPATH)/bin"

# Verify
which protoc-gen-go
which protoc-gen-go-grpc
```

Create New Project

```
mkdir book-catalog-grpc
cd book-catalog-grpc
go mod init book-catalog-grpc

# Install gRPC dependencies
go get google.golang.org/grpc
go get google.golang.org/protobuf
go get github.com/mattn/go-sqlite3
```

📚 Study Materials

Before starting the tasks, make sure to read through the **Lab_Sample_Guide** file which contains:

- Protocol Buffers fundamentals
- Simple gRPC service example
- Complete CRUD service with database
- Error handling patterns
- Common mistakes and solutions

✍️ Task 1: Protocol Buffers Basics (15 points)

🎯 Real-World Application

Every major tech company uses Protocol Buffers! Google uses it for all internal services (millions of RPC calls per second). Uber built their entire microservice architecture on gRPC. Netflix uses it for service-to-service communication. Protocol Buffers are 3-10x smaller than JSON and 20-100x faster to parse. This task teaches you the foundation that powers the world's largest distributed systems!

Challenge: Define Protocol Buffer messages for a book catalog system.

Instructions:

1. **Create `book.proto` file:**

```
syntax = "proto3";

package bookstore;

option go_package = "./proto";

// TODO: Define Book message
// Fields: id (int32), title (string), author (string),
//         isbn (string), price (float), stock (int32),
//         published_year (int32)

// TODO: Define BookCategory enum
// Values: UNKNOWN (0), FICTION (1), NONFICTION (2),
//         SCIFI (3), FANTASY (4), MYSTERY (5), BIOGRAPHY (6)

// TODO: Define DetailedBook message
// Fields: book (Book), category (BookCategory),
//         description (string), tags (repeated string),
//         rating (float)

// TODO: Define Author message
// Fields: id (int32), name (string), bio (string),
//         birth_year (int32), books (repeated Book)
```

2. **Compile the .proto file**

3. **Create `main.go` to test:**
   - Create a Book instance
   - Create a DetailedBook with category and tags
   - Create an Author with multiple books
   - Marshal to bytes (serialize)
   - Unmarshal from bytes (deserialize)
   - Print sizes and values

Starter Code:

```
package main

import (
        "fmt"
        "log"

        pb "./proto"
        "google.golang.org/protobuf/proto"
)

func main() {
        // TODO: Create a Book
        book := &pb.Book{
                // Fill in fields
        }

        fmt.Printf("Book: %v\n", book)

        // TODO: Create DetailedBook with category and tags
        detailedBook := &pb.DetailedBook{
                // Fill in fields
        }

        fmt.Printf("\nDetailed Book: %v\n", detailedBook)
        fmt.Printf("Category: %s\n", detailedBook.Category)
        fmt.Printf("Tags: %v\n", detailedBook.Tags)

        // TODO: Serialize to bytes
        data, err := proto.Marshal(book)
        if err != nil {
                log.Fatal(err)
        }

        fmt.Printf("\nSerialized size: %d bytes\n", len(data))

        // TODO: Deserialize from bytes
        newBook := &pb.Book{}
        err = proto.Unmarshal(data, newBook)
        if err != nil {
                log.Fatal(err)
        }

        fmt.Printf("Deserialized book: %v\n", newBook)

        // TODO: Create Author with multiple books
        author := &pb.Author{
                // Fill in fields with at least 2 books
        }

        fmt.Printf("\nAuthor: %s\n", author.Name)
        fmt.Printf("Books written: %d\n", len(author.Books))
        for i, b := range author.Books {
                fmt.Printf("  %d. %s\n", i+1, b.Title)
        }
}
```

Compile Command:

```
# Create proto directory
mkdir -p proto

# Compile
protoc --go_out=. --go_opt=paths=source_relative book.proto

# This should generate: book.pb.go
```

Run:

```
go run main.go
```

Expected Output:

```
Book: id:1 title:"The Go Programming Language" author:"Alan Donovan" isbn:"978-0134

Detailed Book: book:{id:1 ...} category:NONFICTION description:"..." tags:"programm
Category: NONFICTION
Tags: [programming go technical]

Serialized size: 65 bytes

Deserialized book: id:1 title:"The Go Programming Language" ...

Author: Robert C. Martin
Books written: 2
  1. Clean Code
  2. Clean Architecture
```

Grading Criteria (15 points):

- ✅ Book message defined correctly (3 points)
- ✅ BookCategory enum defined (2 points)
- ✅ DetailedBook message with nested Book (3 points)
- ✅ Author message with repeated Books (2 points)
- ✅ Code compiles successfully (2 points)
- ✅ Serialization/deserialization works (3 points)

✍️ Task 2: Simple gRPC Service (20 points)

🎯 Real-World Application

Microservices need to talk to each other! When you search on Google, dozens of services communicate via gRPC - search service talks to ad service, recommendation service, user profile service, all in milliseconds. Discord processes billions of messages using gRPC. This task teaches you the fundamental pattern for building microservices that can scale to billions of requests!

Challenge: Build a simple Calculator gRPC service to understand RPC fundamentals.

Instructions:

1. **Create `calculator.proto`:**

```
syntax = "proto3";

package calculator;

option go_package = "./proto";

message CalculateRequest {
  float a = 1;
  float b = 2;
  string operation = 3;  // "add", "subtract", "multiply", "divide"
}

message CalculateResponse {
  float result = 1;
  string operation = 2;
}

message SquareRootRequest {
  float number = 1;
}

message SquareRootResponse {
  float result = 1;
}

message HistoryRequest {
  // Empty - get all history
}

message HistoryResponse {
  repeated string calculations = 1;
  int32 count = 2;
}

service Calculator {
  rpc Calculate(CalculateRequest) returns (CalculateResponse);
  rpc SquareRoot(SquareRootRequest) returns (SquareRootResponse);
  rpc GetHistory(HistoryRequest) returns (HistoryResponse);
}
```

2. **Implement Server (`server/main.go`):**
   - Implement Calculate method (add, subtract, multiply, divide)
   - Implement SquareRoot method
   - Implement GetHistory method
   - Store calculation history in memory
   - Handle division by zero error
   - Handle negative square root error

3. **Implement Client (`client/main.go`):**
   - Test all operations
   - Test error cases
   - Get and print history

Starter Code (Server):

```
package main

import (
        "context"
        "fmt"
        "log"
        "math"
        "net"

        pb "../proto"
        "google.golang.org/grpc"
        "google.golang.org/grpc/codes"
        "google.golang.org/grpc/status"
)

type calculatorServer struct {
        pb.UnimplementedCalculatorServer
        history []string  // Store calculation history
}

func (s *calculatorServer) Calculate(ctx context.Context, req *pb.CalculateRequest)
        log.Printf("Calculate: %.2f %s %.2f", req.A, req.Operation, req.B)

        var result float32

        // TODO: Implement operations
        switch req.Operation {
        case "add":
                // TODO: result = a + b
        case "subtract":
                // TODO: result = a - b
        case "multiply":
                // TODO: result = a * b
        case "divide":
                // TODO: Check if b == 0, return error if so
                // TODO: result = a / b
        default:
                return nil, status.Errorf(codes.InvalidArgument,
                        "unknown operation: %s", req.Operation)
        }

        // TODO: Add to history
        historyEntry := fmt.Sprintf("%.2f %s %.2f = %.2f",
                req.A, req.Operation, req.B, result)
        s.history = append(s.history, historyEntry)

        return &pb.CalculateResponse{
                Result:    result,
                Operation: req.Operation,
        }, nil
}

func (s *calculatorServer) SquareRoot(ctx context.Context, req *pb.SquareRootReques
        log.Printf("SquareRoot: %.2f", req.Number)

        // TODO: Check if number is negative
        if req.Number < 0 {
                return nil, status.Errorf(codes.InvalidArgument,
                        "cannot calculate square root of negative number: %.2f", req
        }

        // TODO: Calculate square root
        result := float32(math.Sqrt(float64(req.Number)))

        // TODO: Add to history
        historyEntry := fmt.Sprintf("sqrt(%.2f) = %.2f", req.Number, result)
        s.history = append(s.history, historyEntry)

        return &pb.SquareRootResponse{Result: result}, nil
}

func (s *calculatorServer) GetHistory(ctx context.Context, req *pb.HistoryRequest)
        log.Println("GetHistory called")

        // TODO: Return history
        return &pb.HistoryResponse{
                Calculations: s.history,
                Count:        int32(len(s.history)),
        }, nil
}

func main() {
        // TODO: Create listener on port 50051

        // TODO: Create gRPC server

        // TODO: Register Calculator service

        // TODO: Start serving
}
```

Starter Code (Client):

```
package main

import (
        "context"
        "fmt"
        "log"
        "time"

        pb "../proto"
        "google.golang.org/grpc"
        "google.golang.org/grpc/credentials/insecure"
        "google.golang.org/grpc/status"
)

func main() {
        // TODO: Connect to server
        conn, err := grpc.Dial("localhost:50051",
                grpc.WithTransportCredentials(insecure.NewCredentials()))
        if err != nil {
                log.Fatalf("Failed to connect: %v", err)
        }
        defer conn.Close()

        client := pb.NewCalculatorClient(conn)
        ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
        defer cancel()

        // Test 1: Addition
        fmt.Println("=== Test 1: Addition ===")
        resp, err := client.Calculate(ctx, &pb.CalculateRequest{
                A:         10,
                B:         5,
                Operation: "add",
        })
        // TODO: Handle error and print result

        // Test 2: Division
        fmt.Println("\n=== Test 2: Division ===")
        // TODO: Test 20 / 4

        // Test 3: Division by zero (should error)
        fmt.Println("\n=== Test 3: Division by Zero ===")
        _, err = client.Calculate(ctx, &pb.CalculateRequest{
                A:         10,
                B:         0,
                Operation: "divide",
        })
        if err != nil {
                st, _ := status.FromError(err)
                fmt.Printf("Expected error: %s\n", st.Message())
        }

        // Test 4: Square root
        fmt.Println("\n=== Test 4: Square Root ===")
        // TODO: Test sqrt(16)

        // Test 5: Negative square root (should error)
        fmt.Println("\n=== Test 5: Negative Square Root ===")
        // TODO: Test sqrt(-4), should error

        // Test 6: Get history
        fmt.Println("\n=== Test 6: History ===")
        // TODO: Get and print all calculations
}
```

Expected Output (Server):

```
🚀 Calculator gRPC server listening on :50051
Calculate: 10.00 add 5.00
Calculate: 20.00 divide 4.00
Calculate: 10.00 divide 0.00
SquareRoot: 16.00
SquareRoot: -4.00
GetHistory called
```

Expected Output (Client):

```
=== Test 1: Addition ===
Result: 10.00 + 5.00 = 15.00

=== Test 2: Division ===
Result: 20.00 / 4.00 = 5.00

=== Test 3: Division by Zero ===
Expected error: cannot divide by zero

=== Test 4: Square Root ===
Result: sqrt(16.00) = 4.00

=== Test 5: Negative Square Root ===
Expected error: cannot calculate square root of negative number: -4.00

=== Test 6: History ===
Calculations: 5
1. 10.00 add 5.00 = 15.00
2. 20.00 divide 4.00 = 5.00
3. sqrt(16.00) = 4.00
```

Grading Criteria (20 points):

- ✅ Proto file defined correctly (3 points)
- ✅ Calculate method with all operations (5 points)
- ✅ Division by zero error handling (3 points)
- ✅ SquareRoot method implemented (3 points)
- ✅ Negative square root error handling (2 points)
- ✅ History tracking works (2 points)
- ✅ Client tests all operations (2 points)

✍️ Task 3: Book Catalog Service with Database (20 points)

🎯 Real-World Application

Real microservices need data persistence! Amazon's product catalog service handles millions of items in databases, accessed via gRPC by checkout service, recommendation service, and search service. Shopify's inventory service uses gRPC to communicate with order processing. This task teaches you to build a production-ready microservice with database integration - the foundation of any e-commerce or content platform!

Challenge: Build a complete Book Catalog gRPC service with SQLite database.

Instructions:

1. **Create `book_service.proto`:**

```
syntax = "proto3";

package bookservice;

option go_package = "./proto";

message Book {
  int32 id = 1;
  string title = 2;
  string author = 3;
  string isbn = 4;
  float price = 5;
  int32 stock = 6;
  int32 published_year = 7;
}

// TODO: Define GetBookRequest (id)
// TODO: Define GetBookResponse (book)

// TODO: Define CreateBookRequest (all fields except id)
// TODO: Define CreateBookResponse (book)

// TODO: Define UpdateBookRequest (all fields including id)
// TODO: Define UpdateBookResponse (book)

// TODO: Define DeleteBookRequest (id)
// TODO: Define DeleteBookResponse (success, message)

// TODO: Define ListBooksRequest (page, page_size)
// TODO: Define ListBooksResponse (books, total, page, page_size)

service BookCatalog {
  rpc GetBook(GetBookRequest) returns (GetBookResponse);
  rpc CreateBook(CreateBookRequest) returns (CreateBookResponse);
  rpc UpdateBook(UpdateBookRequest) returns (UpdateBookResponse);
  rpc DeleteBook(DeleteBookRequest) returns (DeleteBookResponse);
  rpc ListBooks(ListBooksRequest) returns (ListBooksResponse);
}
```

2. **Implement Server:**
   - Initialize SQLite database
   - Seed sample data (at least 5 books)
   - Implement all RPC methods
   - Handle errors appropriately
   - Use context for database operations

3. **Implement Client:**
   - Test all operations
   - Demonstrate CRUD cycle
   - Test pagination

Starter Code (Server):

```
package main

import (
        "context"
        "database/sql"
        "fmt"
        "log"
        "net"

        pb "../proto"
        _ "github.com/mattn/go-sqlite3"
        "google.golang.org/grpc"
        "google.golang.org/grpc/codes"
        "google.golang.org/grpc/status"
)
```

```
type bookCatalogServer struct {
	pb.UnimplementedBookCatalogServer
	db *sql.DB
}

func (s *bookCatalogServer) GetBook(ctx context.Context, req *pb.GetBookRequest) (*p
	// TODO: Query book from database
	// TODO: Handle not found case
	// TODO: Return book
	return nil, status.Error(codes.Unimplemented, "not implemented")
}

func (s *bookCatalogServer) CreateBook(ctx context.Context, req *pb.CreateBookReques
	// TODO: Validate input
	// TODO: Insert into database
	// TODO: Return created book with ID
	return nil, status.Error(codes.Unimplemented, "not implemented")
}

func (s *bookCatalogServer) UpdateBook(ctx context.Context, req *pb.UpdateBookReques
	// TODO: Validate input
	// TODO: Update in database
	// TODO: Check if exists (RowsAffected)
	// TODO: Return updated book
	return nil, status.Error(codes.Unimplemented, "not implemented")
}

func (s *bookCatalogServer) DeleteBook(ctx context.Context, req *pb.DeleteBookReques
	// TODO: Delete from database
	// TODO: Check if exists
	// TODO: Return success response
	return nil, status.Error(codes.Unimplemented, "not implemented")
}

func (s *bookCatalogServer) ListBooks(ctx context.Context, req *pb.ListBooksRequest)
	// TODO: Implement pagination
	// TODO: Get total count
	// TODO: Query books with LIMIT/OFFSET
	// TODO: Return response
	return nil, status.Error(codes.Unimplemented, "not implemented")
}

func initDB() (*sql.DB, error) {
	db, err := sql.Open("sqlite3", "./books.db")
	if err != nil {
		return nil, err
	}

	// TODO: Create books table
	// TODO: Seed sample data (5+ books)

	return db, nil
}

func main() {
	// TODO: Initialize database

	// TODO: Create listener

	// TODO: Create gRPC server

	// TODO: Register service

	// TODO: Start serving
}
```

Expected Client Output:

```
=== Test 1: List All Books ===
Total books: 5
1. The Go Programming Language by Alan Donovan - $39.99
2. Clean Code by Robert Martin - $42.50
...

=== Test 2: Get Book ===
Book ID: 1
Title: The Go Programming Language
Author: Alan Donovan
Price: $39.99

=== Test 3: Create Book ===
Created book ID: 6
Title: Learning Go

=== Test 4: Update Book ===
Updated book: The Go Programming Language (2nd Edition)
New price: $35.99

=== Test 5: Delete Book ===
Book deleted successfully

=== Test 6: Pagination ===
Page 1: 3 books
Page 2: 2 books
```

Grading Criteria (20 points):

- ✅ Proto service defined correctly (3 points)
- ✅ Database initialization with seed data (2 points)
- ✅ GetBook implemented (3 points)
- ✅ CreateBook implemented (3 points)
- ✅ UpdateBook implemented (3 points)
- ✅ DeleteBook implemented (2 points)
- ✅ ListBooks with pagination (4 points)

✍️ Task 4: Advanced Search and Error Handling (20 points)

🎯 Real-World Application

Search is critical for any catalog! Amazon's search handles 2 billion searches per day via microservices. Netflix search service uses gRPC to find content across hundreds of millions of titles. Proper error handling prevents cascading failures - when one service fails, it shouldn't crash everything. This task teaches you production-grade search and resilience patterns used by major platforms!

Challenge: Add search functionality and comprehensive error handling to your Book Catalog service.

Instructions:

1. **Extend `book_service.proto`:**

```
// Add to existing book_service.proto

message SearchBooksRequest {
  string query = 1;
  string field = 2;  // "title", "author", "isbn", or "all"
}

message SearchBooksResponse {
  repeated Book books = 1;
  int32 count = 2;
  string query = 3;
}

message FilterBooksRequest {
  float min_price = 1;
  float max_price = 2;
  int32 min_year = 3;
  int32 max_year = 4;
}

message FilterBooksResponse {
  repeated Book books = 1;
  int32 count = 2;
}

message GetStatsRequest {
  // Empty
}

message GetStatsResponse {
  int32 total_books = 1;
  float average_price = 2;
  int32 total_stock = 3;
  int32 earliest_year = 4;
  int32 latest_year = 5;
}

// Add to service
service BookCatalog {
  // ... existing methods ...

  rpc SearchBooks(SearchBooksRequest) returns (SearchBooksResponse);
  rpc FilterBooks(FilterBooksRequest) returns (FilterBooksResponse);
  rpc GetStats(GetStatsRequest) returns (GetStatsResponse);
}
```

2. **Implement Server Methods:**
   - SearchBooks: Search by title, author, ISBN, or all fields
   - FilterBooks: Filter by price range and year range
   - GetStats: Calculate statistics (count, avg price, etc.)
   - Add proper validation and error handling
   - Return appropriate gRPC status codes

3. **Implement Error Handling:**
   - Validate all input parameters
   - Return InvalidArgument for bad input
   - Return NotFound when appropriate
   - Return Internal for database errors
   - Include helpful error messages

4. **Create Client Tests:**
   - Test search with different fields
   - Test filter with various ranges
   - Test stats calculation
   - Test error cases (empty query, invalid ranges)

Starter Code (Server):

```
func (s *bookCatalogServer) SearchBooks(ctx context.Context, req *pb.SearchBooksRequ
	log.Printf("SearchBooks: query=%s, field=%s", req.Query, req.Field)

	// TODO: Validate input
	if req.Query == "" {
		return nil, status.Error(codes.InvalidArgument, "search query requir
	}

	// TODO: Build SQL query based on field
	var sqlQuery string
	var args []interface{}

	searchPattern := "%" + req.Query + "%"

	switch req.Field {
	case "title":
		// TODO: Search only in title
		sqlQuery = "SELECT id, title, author, isbn, price, stock, published_
		args = []interface{}{searchPattern}
	case "author":
		// TODO: Search only in author
	case "isbn":
		// TODO: Search in ISBN (exact match)
	case "all", "":
		// TODO: Search in all fields
		sqlQuery = `SELECT id, title, author, isbn, price, stock, published_
			FROM books
			WHERE title LIKE ? OR author LIKE ? OR isbn LIKE ?`
		args = []interface{}{searchPattern, searchPattern, searchPattern}
	default:
		return nil, status.Errorf(codes.InvalidArgument, "invalid field: %s"
	}

	// TODO: Execute query
	// TODO: Scan results
	// TODO: Return response

	return nil, status.Error(codes.Unimplemented, "not implemented")
}

func (s *bookCatalogServer) FilterBooks(ctx context.Context, req *pb.FilterBooksRequ
	log.Printf("FilterBooks: price[%.2f-%.2f], year[%d-%d]",
		req.MinPrice, req.MaxPrice, req.MinYear, req.MaxYear)

	// TODO: Validate ranges
	if req.MinPrice < 0 || req.MaxPrice < 0 {
		return nil, status.Error(codes.InvalidArgument, "price cannot be neg
	}
	if req.MinPrice > req.MaxPrice && req.MaxPrice > 0 {
		return nil, status.Error(codes.InvalidArgument, "min_price cannot be
	}

	// TODO: Build dynamic query
	query := "SELECT id, title, author, isbn, price, stock, published_year FROM
	var args []interface{}

	// TODO: Add price filters if provided
	if req.MinPrice > 0 {
		query += " AND price >= ?"
		args = append(args, req.MinPrice)
	}
	// TODO: Add max_price, min_year, max_year filters

	// TODO: Execute query
	// TODO: Return results

	return nil, status.Error(codes.Unimplemented, "not implemented")
}

func (s *bookCatalogServer) GetStats(ctx context.Context, req *pb.GetStatsRequest) (
	log.Println("GetStats called")

	var stats pb.GetStatsResponse

	// TODO: Get total books
	err := s.db.QueryRowContext(ctx, "SELECT COUNT(*) FROM books").Scan(&stats.T
	if err != nil {
		return nil, status.Errorf(codes.Internal, "failed to count books: %v
	}

	// TODO: Get average price
	// TODO: Get total stock
	// TODO: Get earliest and latest year

	return &stats, nil
}
```

Expected Output:

```
=== Test 1: Search by Title ===
Searching for "go"...
Found 2 books:
- The Go Programming Language
- Learning Go

=== Test 2: Search by Author ===
Searching for "Martin"...
Found 2 books:
- Clean Code by Robert Martin
- Clean Architecture by Robert Martin

=== Test 3: Filter by Price ===
Books between $20 and $45:
Found 3 books

=== Test 4: Filter by Year ===
Books published after 2010:
Found 4 books

=== Test 5: Get Statistics ===
Total books: 6
Average price: $42.45
Total stock: 95
Year range: 1994 - 2021

=== Test 6: Error Cases ===
Empty search query: Error: search query required
Invalid price range: Error: min_price cannot be greater than max_price
```

Grading Criteria (20 points):

- ✅ SearchBooks implemented (5 points)
- ✅ Search by specific field works (3 points)
- ✅ FilterBooks implemented (4 points)
- ✅ GetStats implemented (3 points)
- ✅ Proper input validation (3 points)
- ✅ Correct gRPC status codes (2 points)

✍️ Task 5: Complete Microservice with Multiple Services (25 points) - REQUIRED

🎯 Real-World Application

Real systems have multiple interconnected services! When you buy on Amazon: inventory service checks stock (gRPC), pricing service calculates total (gRPC), payment service processes card (gRPC), shipping service creates label (gRPC) - all coordinated via microservices. This task teaches you to build a multi-service architecture that mirrors production systems at scale!

Challenge: Build a complete microservice system with Book Catalog and Author services that communicate with each other.

Instructions:

1. **Create `author_service.proto`:**

```
syntax = "proto3";

package authorservice;

option go_package = "./proto";

message Author {
  int32 id = 1;
  string name = 2;
  string bio = 3;
  int32 birth_year = 4;
  string country = 5;
}

message GetAuthorRequest {
  int32 id = 1;
}

message GetAuthorResponse {
  Author author = 1;
}

message CreateAuthorRequest {
  string name = 1;
  string bio = 2;
  int32 birth_year = 3;
  string country = 4;
}

message CreateAuthorResponse {
  Author author = 1;
}

message ListAuthorsRequest {
  int32 page = 1;
  int32 page_size = 2;
}

message ListAuthorsResponse {
  repeated Author authors = 1;
  int32 total = 2;
}

message GetAuthorBooksRequest {
  int32 author_id = 1;
}

// Reference to Book from book_service
message BookSummary {
  int32 id = 1;
  string title = 2;
  float price = 3;
  int32 published_year = 4;
}

message GetAuthorBooksResponse {
  Author author = 1;
  repeated BookSummary books = 2;
  int32 book_count = 3;
}

service AuthorCatalog {
  rpc GetAuthor(GetAuthorRequest) returns (GetAuthorResponse);
  rpc CreateAuthor(CreateAuthorRequest) returns (CreateAuthorResponse);
  rpc ListAuthors(ListAuthorsRequest) returns (ListAuthorsResponse);
  rpc GetAuthorBooks(GetAuthorBooksRequest) returns (GetAuthorBooksResponse);
}
```

2. **Update Book Service:**
   - Add `author_id` field to Book message
   - Create `GetBooksByAuthor` RPC method
   - Implement foreign key relationship

3. **Implement Author Service:**
   - CRUD operations for authors
   - GetAuthorBooks method that calls Book service
   - Database with authors table

4. **Create Service Communication:**
   - Author service connects to Book service as client
   - Demonstrates service-to-service communication
   - Handles inter-service errors

5. **Build Demo Client:**
   - Connect to both services
   - Create author
   - Create books for that author
   - Fetch author's books (demonstrates cross-service call)
   - Show statistics from both services

Starter Code (Author Service):

```
package main

import (
	"context"
	"database/sql"
	"log"
	"net"

	authorpb "../proto/author"
	bookpb "../proto/book"
	_ "github.com/mattn/go-sqlite3"
	"google.golang.org/grpc"
	"google.golang.org/grpc/credentials/insecure"
	"google.golang.org/grpc/codes"
	"google.golang.org/grpc/status"
)

type authorCatalogServer struct {
	authorpb.UnimplementedAuthorCatalogServer
	db         *sql.DB
	bookClient bookpb.BookCatalogClient  // Client to Book service
}

func newServer(db *sql.DB, bookClient bookpb.BookCatalogClient) *authorCatalogServer
	return &authorCatalogServer{
		db:         db,
		bookClient: bookClient,
	}
}

func (s *authorCatalogServer) GetAuthor(ctx context.Context, req *authorpb.GetAuthor
	// TODO: Implement GetAuthor
	return nil, status.Error(codes.Unimplemented, "not implemented")
}

func (s *authorCatalogServer) CreateAuthor(ctx context.Context, req *authorpb.Create
	// TODO: Implement CreateAuthor
	return nil, status.Error(codes.Unimplemented, "not implemented")
}

func (s *authorCatalogServer) ListAuthors(ctx context.Context, req *authorpb.ListAut
	// TODO: Implement ListAuthors with pagination
	return nil, status.Error(codes.Unimplemented, "not implemented")
}

func (s *authorCatalogServer) GetAuthorBooks(ctx context.Context, req *authorpb.GetA
	log.Printf("GetAuthorBooks: author_id=%d", req.AuthorId)

	// TODO: Get author from database
	var author authorpb.Author
	err := s.db.QueryRowContext(ctx,
		"SELECT id, name, bio, birth_year, country FROM authors WHERE id = ?
		req.AuthorId,
	).Scan(&author.Id, &author.Name, &author.Bio, &author.BirthYear, &author.Cou

	if err == sql.ErrNoRows {
		return nil, status.Errorf(codes.NotFound, "author not found: id=%d",
	}
	if err != nil {
		return nil, status.Errorf(codes.Internal, "database error: %v", err)
	}

	// TODO: Call Book service to get books by this author
	// This demonstrates service-to-service communication!
	bookResp, err := s.bookClient.GetBooksByAuthor(ctx, &bookpb.GetBooksByAuthor
		AuthorId: req.AuthorId,
	})
	if err != nil {
		log.Printf("Failed to get books: %v", err)
		// Continue even if book service fails
		return &authorpb.GetAuthorBooksResponse{
			Author:    &author,
			Books:     nil,
			BookCount: 0,
		}, nil
	}

	// TODO: Convert books to BookSummary
	var bookSummaries []*authorpb.BookSummary
	for _, book := range bookResp.Books {
		bookSummaries = append(bookSummaries, &authorpb.BookSummary{
			Id:            book.Id,
			Title:         book.Title,
			Price:         book.Price,
			PublishedYear: book.PublishedYear,
		})
	}

	return &authorpb.GetAuthorBooksResponse{
		Author:    &author,
		Books:     bookSummaries,
		BookCount: int32(len(bookSummaries)),
	}, nil
}

func connectToBookService() (bookpb.BookCatalogClient, error) {
	// TODO: Connect to Book service
	conn, err := grpc.Dial("localhost:50051",
		grpc.WithTransportCredentials(insecure.NewCredentials()))
	if err != nil {
		return nil, err
	}

	return bookpb.NewBookCatalogClient(conn), nil
}

func initDB() (*sql.DB, error) {
	db, err := sql.Open("sqlite3", "./authors.db")
	if err != nil {
		return nil, err
	}

	// TODO: Create authors table
	// TODO: Seed sample authors

	return db, nil
}

func main() {
	// TODO: Initialize database
	db, err := initDB()
	if err != nil {
		log.Fatalf("Failed to init DB: %v", err)
	}
	defer db.Close()

	// TODO: Connect to Book service
	bookClient, err := connectToBookService()
	if err != nil {
		log.Fatalf("Failed to connect to Book service: %v", err)
	}

	// TODO: Create listener on different port (50052)
	lis, err := net.Listen("tcp", ":50052")
	if err != nil {
		log.Fatalf("Failed to listen: %v", err)
	}

	// TODO: Create gRPC server
	grpcServer := grpc.NewServer()

	// TODO: Register service
	authorpb.RegisterAuthorCatalogServer(grpcServer, newServer(db, bookClient))

	log.Println("🚀 Author Catalog gRPC server listening on :50052")
	log.Println("📚 Connected to Book Catalog service on :50051")

	// TODO: Start serving
	if err := grpcServer.Serve(lis); err != nil {
		log.Fatalf("Failed to serve: %v", err)
	}
}
```

Demo Client:

```
package main

import (
	"context"
	"fmt"
	"log"
	"time"

	authorpb "../proto/author"
	bookpb "../proto/book"
	"google.golang.org/grpc"
	"google.golang.org/grpc/credentials/insecure"
)

func main() {
	// Connect to both services
	bookConn, err := grpc.Dial("localhost:50051",
		grpc.WithTransportCredentials(insecure.NewCredentials()))
	if err != nil {
		log.Fatal(err)
	}
	defer bookConn.Close()

	authorConn, err := grpc.Dial("localhost:50052",
		grpc.WithTransportCredentials(insecure.NewCredentials()))
	if err != nil {
		log.Fatal(err)
	}
	defer authorConn.Close()

	bookClient := bookpb.NewBookCatalogClient(bookConn)
	authorClient := authorpb.NewAuthorCatalogClient(authorConn)

	ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
	defer cancel()

	fmt.Println("=== Microservice Demo ===\n")

	// 1. Create author
	fmt.Println("1. Creating author...")
	authorResp, err := authorClient.CreateAuthor(ctx, &authorpb.CreateAuthorReque
		Name:      "Martin Fowler",
		Bio:       "Software development expert",
		BirthYear: 1963,
		Country:   "UK",
	})
	if err != nil {
		log.Printf("Failed to create author: %v", err)
	} else {
		fmt.Printf("✓ Created author: %s (ID: %d)\n\n",
			authorResp.Author.Name, authorResp.Author.Id)

		// 2. Create books for this author
		fmt.Println("2. Creating books for author...")
		book1, _ := bookClient.CreateBook(ctx, &bookpb.CreateBookRequest{
			Title:         "Refactoring",
			AuthorId:      authorResp.Author.Id,
			Isbn:          "978-0134757599",
			Price:         49.99,
			Stock:         15,
			PublishedYear: 2018,
		})
		fmt.Printf("✓ Created book: %s\n", book1.Book.Title)

		book2, _ := bookClient.CreateBook(ctx, &bookpb.CreateBookRequest{
			Title:         "Patterns of Enterprise Application Architectu
			AuthorId:      authorResp.Author.Id,
			Isbn:          "978-0321127426",
			Price:         54.99,
			Stock:         8,
			PublishedYear: 2002,
		})
		fmt.Printf("✓ Created book: %s\n\n", book2.Book.Title)

		// 3. Get author's books (cross-service call)
		fmt.Println("3. Fetching author's books (cross-service call)...")
		booksResp, err := authorClient.GetAuthorBooks(ctx, &authorpb.GetAuth
			AuthorId: authorResp.Author.Id,
		})
		if err != nil {
			log.Printf("Failed: %v", err)
		} else {
			fmt.Printf("✓ Author: %s\n", booksResp.Author.Name)
			fmt.Printf("✓ Books written: %d\n", booksResp.BookCount)
			for i, book := range booksResp.Books {
				fmt.Printf("  %d. %s (%d)\n", i+1, book.Title, book.P
			}
		}
	}

	fmt.Println("\n✓ Microservice demo completed!")
}
```

Expected Output:

```
=== Microservice Demo ===

1. Creating author...
✓ Created author: Martin Fowler (ID: 4)

2. Creating books for author...
✓ Created book: Refactoring
✓ Created book: Patterns of Enterprise Application Architecture

3. Fetching author's books (cross-service call)...
✓ Author: Martin Fowler
✓ Books written: 2
✓ Books written: 2
  1. Refactoring (2018)
  2. Patterns of Enterprise Application Architecture (2002)

✓ Microservice demo completed!
```

Grading Criteria (25 points):

- ✅ Author service proto defined (3 points)
- ✅ Author CRUD operations (5 points)
- ✅ Book service updated with author_id (3 points)
- ✅ GetBooksByAuthor implemented (4 points)
- ✅ GetAuthorBooks with service-to-service call (6 points)
- ✅ Both services run simultaneously (2 points)
- ✅ Demo client shows integration (2 points)

Submission Requirements (100 points)

Tasks (85 points)

- ☐ Task 1: Protocol Buffers Basics (15 points)
- ☐ Task 2: Simple gRPC Service (20 points)
- ☐ Task 3: Book Catalog Service with Database (20 points)
- ☐ Task 4: Advanced Search and Error Handling (20 points)
- ☐ Task 5: Complete Microservice System (25 points) - **REQUIRED**

Code Quality (15 points)

- ☐ Code compiles and runs (5 points)
- ☐ Proper error handling with status codes (4 points)
- ☐ Clean, readable code with comments (3 points)
- ☐ Context used appropriately (3 points)

Bonus (+15 points)

- ☐ Implement interceptor for logging (+4 points)
- ☐ Add timeout and retry logic (+4 points)
- ☐ Implement server-side streaming (+4 points)
- ☐ Add gRPC metadata (headers) (+3 points)

Testing Checklist

Before submitting, verify:

1. ✓ All .proto files compile successfully
2. ✓ Generated .pb.go files are present
3. ✓ All servers start without errors
4. ✓ All clients can connect to servers
5. ✓ CRUD operations work correctly
6. ✓ Error handling returns correct status codes
7. ✓ Service-to-service communication works
8. ✓ Database operations use context
9. ✓ Connections are properly closed
10. ✓ No panics or crashes during testing

Connection to Final Project (MangaHub)

This lab directly prepares you for the MangaHub project by teaching transferable patterns:

What You Practiced (Books):

1. **Task 2**: Calculator service → **Apply to**: Utility services
2. **Task 3**: Book CRUD → **Apply to**: Manga CRUD via gRPC
3. **Task 4**: Search/filter → **Apply to**: Manga search service
4. **Task 5**: Multi-service → **Apply to**: Manga + User + Reading services

Direct Skill Transfer:

Lab Pattern (Books)	Project Application (Manga)
Book Catalog service	Manga Catalog service
Author service	Artist/Author service
Search books	Search manga
Book-Author relation	Manga-Artist relation
Service-to-service calls	Microservice communication
gRPC error codes	Consistent error handling
Database with gRPC	Manga database service

Code Patterns You Can Reuse:

```
// Lab Pattern (Books)
service BookCatalog {
  rpc GetBook(GetBookRequest) returns (GetBookResponse);
  rpc CreateBook(CreateBookRequest) returns (CreateBookResponse);
}

// Project Pattern (Manga) - Same structure!
service MangaCatalog {
  rpc GetManga(GetMangaRequest) returns (GetMangaResponse);
  rpc CreateManga(CreateMangaRequest) returns (CreateMangaResponse);
}
```

What Changes for MangaHub:

1. **Message Names**: Book → Manga, Author → Artist
2. **Fields**: ISBN → MangaID, Price → Rating
3. **Additional Services**: ReadingProgress, UserPreferences
4. **Everything Else**: Same gRPC patterns, same architecture!

MangaHub Architecture:

```
┌─────────────┐      gRPC      ┌─────────────┐
│  REST API   │ ←────────────→ │ Manga       │
│  (HTTP)     │                │ Service     │
└─────────────┘                └─────────────┘
                                      │ gRPC
                                      ↓
                               ┌─────────────┐
                               │ Artist      │
                               │ Service     │
                               └─────────────┘
```

Next Steps for Your Project:

1. ✅ Complete all lab tasks with books
2. 🔄 Copy service structure
3. 🔄 Replace Book messages with Manga messages
4. 🔄 Replace Author service with Artist service
5. ➕ Add ReadingProgress service
6. ➕ Add UserPreferences service
7. ➕ Connect all via gRPC

You now have the microservice foundation for MangaHub!

Getting Help

During Lab:

- 🤖 Ask AI: "How do I define protobuf service?" or "gRPC error handling example"
- 👥 Ask classmates
- 🧑‍🏫 Ask instructor
- 📚 Check the Sample Guide (Lab_Sample_Guide)

Useful Prompts for AI:

- "Protocol Buffers message definition example"
- "gRPC service implementation in Go"
- "How to call gRPC service from another service"
- "gRPC status codes for error handling"
- "Context usage in gRPC Go"
- "How to compile .proto files"

Resources:

- gRPC Documentation: https://grpc.io/docs/languages/go/
- Protocol Buffers: https://protobuf.dev/
- gRPC Go Tutorial: https://grpc.io/docs/languages/go/quickstart/
- Status Codes: https://grpc.github.io/grpc/core/md_doc_statuscodes.html

---

Common Protobuf Commands Reference

```
# Compile single proto file
protoc --go_out=. --go_opt=paths=source_relative \
       --go-grpc_out=. --go-grpc_opt=paths=source_relative \
       file.proto

# Compile multiple files
protoc --go_out=. --go_opt=paths=source_relative \
       --go-grpc_out=. --go-grpc_opt=paths=source_relative \
       *.proto

# Compile with specific output directory
protoc --go_out=./generated --go_opt=paths=source_relative \
       --go-grpc_out=./generated --go-grpc_opt=paths=source_relative \
       file.proto

# Check proto syntax
protoc --descriptor_set_out=/dev/null file.proto
```

End of Lab Tasks